A Simple Blueprint for Building Software with AI Agents

Stealing the best of Spec Kit, Superpowers and GStack into one workflow a data scientist can actually follow, from idea to shipped.

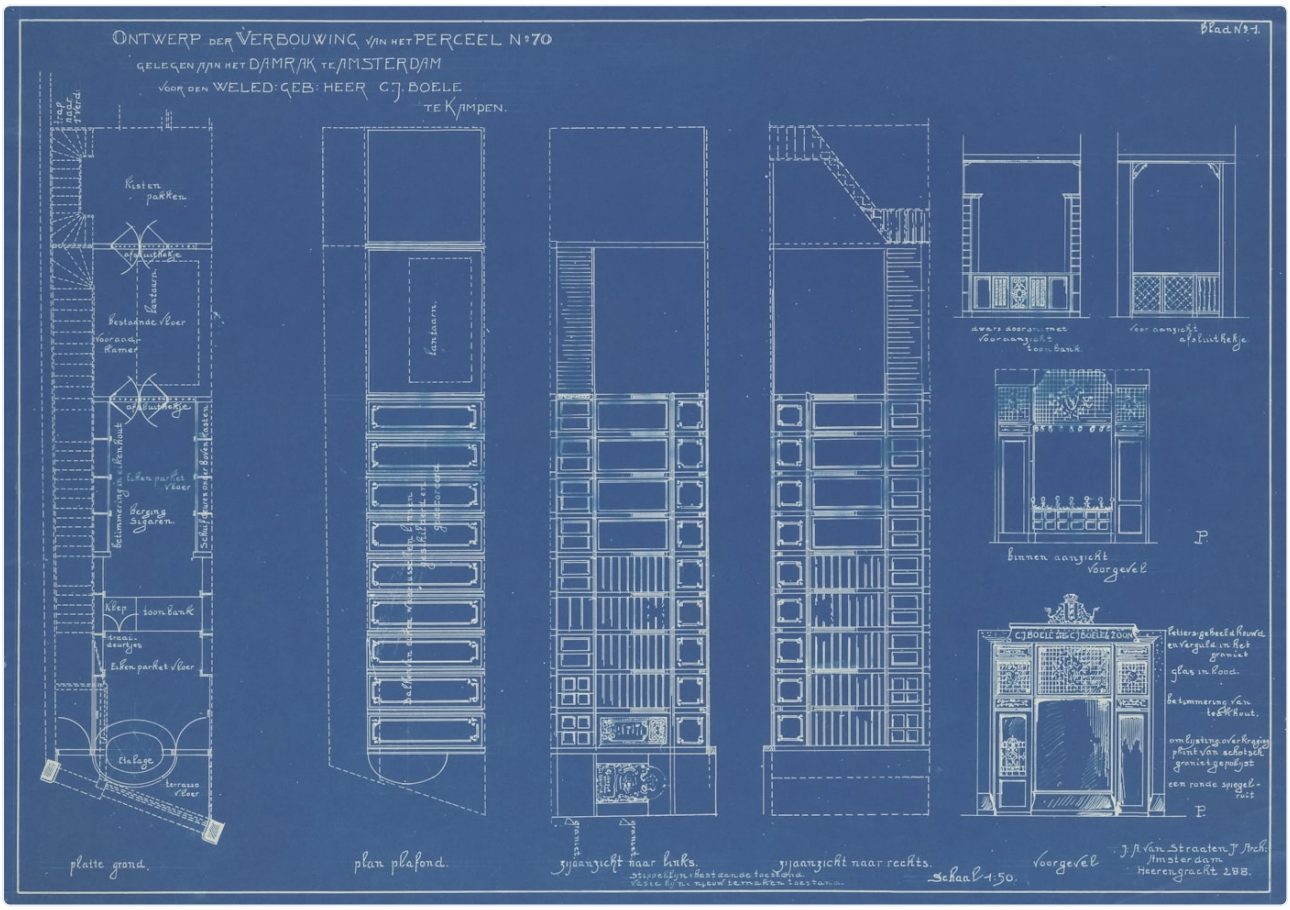

Photo by Amsterdam City Archives on Unsplash

I'll be honest about where I'm coming from. I've spent years in data science, and for most of that time my "development process" was a notebook, a lot of cells run out of order, and faith that I'd remember what I did last Tuesday. It works right up until the thing you built has to survive being used by someone who isn't you. Data scientists learn to model. We rarely learn the software development lifecycle around the model: the spec, the plan, the review, the tests, the boring paperwork that stops a Friday deploy becoming a Saturday incident.

Then AI coding agents arrived, and with them a wave of frameworks that bolt a real engineering process onto the agent. Three kept coming up in my setup: **Spec Kit**, **Superpowers**, and **GStack**. Each is good. Between them they also carry something like sixty slash-commands, which is a lot to stare at on a Monday. So I did what I do with any new tool: tried them, got confused, and distilled the confusion into one blueprint I could remember. The

remarks below are my personal take basis usage, not a benchmark, and there's no promotion here.

Image by Author

Vibe coding got us here, and where it stops

Andrej Karpathy coined "vibe coding" in early 2025: you give in to the vibes and, in his words,

"'Accept All' always, I don't read the diffs anymore."

For a weekend throwaway it's genuinely great. The trouble starts on the last stretch. Addy Osmani calls it the 70% problem: the agent sprints you to 70 percent, then the final 30 becomes "whack-a-mole with code you don't fully understand." Simon Willison's definition lands it, vibe coding is "generating code with AI without caring about the code that is produced." Fine when it's just you; risky the moment it touches production or a teammate.

The fix isn't to stop using agents. It's to give the agent a

document

to build against instead of a vibe, so every change is small and reviewable. That is the whole point of the blueprint below.

A one-minute primer on the three toolkits

- **Spec Kit** (GitHub): the planning engine. It turns an idea into readable Markdown, `spec.md` then `plan.md` then `tasks.md`, so the agent codes against a document.
- **GStack**: the review-and-ship engine. A command for every late stage: `/office-hours`, `/plan-eng-review`, `/review`, `/cso`, `/qa`, `/ship`, `/canary`.
- **Superpowers** (obra): the discipline. Real test-driven development and an append-only verification log. It's actually the most-starred of the three (roughly 243k stars to Spec Kit's 117k), but it bakes its method into agent behaviour and runs only on Claude. So I anchor on Spec Kit for the method itself, because a newcomer learns faster when the method lives in files you can read, and lean on Superpowers for how the building actually gets done: test first, and prove it afterwards.

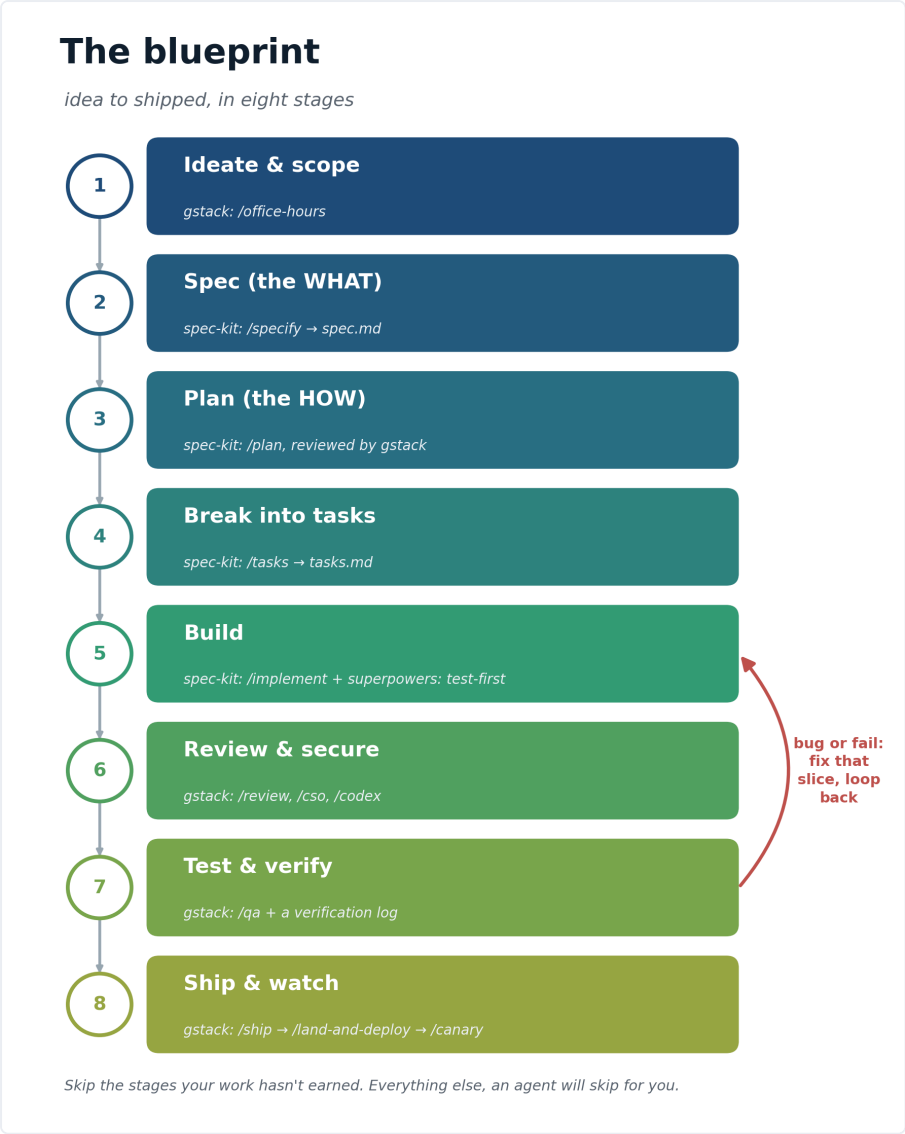

Image by Author

The blueprint, stage by stage

Pick the stages your work has actually earned. A throwaway script needs almost none of this; anything another human will touch needs most of it.

1. Ideate and scope. Before any code, ask whether the thing is worth building. GStack's `/office-hours` challenges the premise and offers alternatives instead of rubber-stamping your first idea. The most expensive code is the code you shouldn't have written.

2. Spec, the WHAT. Spec Kit's `/specify` writes `spec.md`: what you're building, success criteria, edge cases, and no implementation detail. `/clarify` then makes you quantify the vague words, so "fast" becomes "under 200ms". This single habit changed my output the most.

3. Plan, the HOW. `/plan` turns the spec into an architecture in `plan.md`. Hand that to GStack's `/plan-eng-review`. Spec Kit writes the plan; GStack asks whether it's any good. Catching a design flaw here, on a Markdown file, is far cheaper than catching it in code.

4. Break it into tasks. `/tasks` splits the plan into `tasks.md`, a list of small, ordered, testable steps. Now both you and the agent always know what "done" means.

5. Build. `/implement` works the task list, and this is where Superpowers earns its place. Two habits worth stealing: write the failing test first (no production code without a test that proves it), and build in small slices, one pull request each, never a 4,000-line blob nobody can review. The trade-off is real, test-first and small PRs are slower to ship and far easier to trust and to review.

6. Review and secure. An agent reviewing its own code has the same blind spot as a student grading their own exam. GStack's `/review` is the gate every diff should pass, and `/codex` adds a genuinely different model's second opinion when the change is worth it. For security, `/cso` runs an OWASP-style audit, but if you're building GenAI products the threats shift under your feet: an agent that ingests external content (user uploads, scraped pages) can be prompt-injected into working for the attacker, a dependency the agent cheerfully suggests can be a typosquat or a supply-chain trap, and an agent with filesystem access is one bad step from leaking a secret. The OWASP Top 10 for LLM Applications is the checklist worth keeping open here. Security is a named stage, not a vague good intention.

7. Test and verify. `/qa` drives a real browser through every route, finds bugs, and fixes them. Then keep the proof, an idea I took from Superpowers: an append-only verification log stamped with the commit, the test, and the result. You append, never edit, so a wrong-then-corrected entry becomes the audit trail. It feels like paperwork until the day someone asks "was this actually tested?" and you answer with a commit hash instead of a shrug. Human review scales badly, so automate the judgment where you can: a regression suite the agent has to keep green, an LLM-as-judge to grade outputs that have no single right answer, and a spec-to-code traceability pass (Spec Kit's `/analyze` is one) that checks every requirement in `spec.md` was actually built. For a GenAI system the eval suite *is* the test suite; skip it and you're shipping on vibes again.

8. Ship and watch. `/ship` opens the PR, `/land-and-deploy` merges and deploys, `/canary` watches production for errors afterwards. Then plan for it going wrong, because sometimes it will. Keep changes forward-only and reversible so a failed canary rolls back to the last green commit in one step, not a panicked 2am hotfix. And when review misses a regression, or a late spec change invalidates tasks you already built, treat it as a loop rather than a restart: fix the

spec, let the dependent tasks fall out, rebuild only those. That is what the arrows looping back in the diagram are for. This unglamorous tail is the part data science training skips entirely.

Using it for everyday work: features and PRs

You rarely start from a blank repo. Most real work is a new feature on an existing codebase, or reviewing someone else's change. The blueprint scales down for both.

For a **new feature**, run the same eight stages in miniature: a short spec for just that feature, a quick plan, a handful of tasks, then build. Spec Kit is designed for this iterative loop, not only greenfield projects, so you get a fresh `spec.md` per feature while the old ones stay behind as a record of why things are the way they are.

For **reviewing a PR**, the "one slice, one PR" habit from stage 5 is what makes review possible at all. A small, single-purpose diff is something a human, or `/review`, or a second model through `/codex`, can actually reason about. Point the same review gate at a teammate's PR, not just your own. A 4,000-line PR does not get reviewed, it gets rubber-stamped.

Where this meets the data science reality

If you came up through data science, one objection should be nagging at you: you can't spec what you haven't explored yet. Half of our work is opening a notebook, poking the data, and finding out whether the idea is even possible. Writing a `spec.md` first would be pretending to know an answer you don't have.

That's real, and it doesn't break the blueprint. It just adds a stage zero. Exploration is a different mode: a deliberately loose, vibe-coded spike whose only output is a decision about what is worth building. The discipline isn't "never explore freely", it's knowing which mode you're in and switching once the data has told you what works. The failure is staying in exploration mode long after you should have written the spec.

The rest of the blueprint has quiet ML cousins you already half-use. The verification log is experiment tracking. Reproducible tasks are pipeline reproducibility. Model and data versioning is the same instinct as writing down what you built and why. Notebook-to-production is just "demo versus ship it for real" wearing an ML hat. Spec-driven building is what turns a promising notebook into something an underwriter, or a user, can actually rely on.

The real value: it forces you to think first

Look back at those eight stages and notice what they share: almost every one exists to make you think and design in detail before a line of code is written. The spec makes you decide what you are building. The plan makes you decide how, on paper, where changing your mind is cheap. Review and security make you look hard before you ship. Left to itself, an agent skips all of it and starts typing. This workflow is the forcing function that makes you slow down and design first, and that discipline, far more than any single command, was the piece I had been missing. It isn't infrastructure or tooling. It's a way of thinking that the tools happen to make cheap.

Learnings

- The process is the point, not the tool. The real unlock was internalising the sequence and the reason for each step; the toolkits just make each step cheap.
- Spec Kit plans, Superpowers keeps the build honest, GStack ships. If you remember one split, remember that.
- Security and verification were my two biggest gaps, and they're exactly the two a modeller's training never covers. Making them named, non-skippable steps did more for me than any amount of better prompting.

If you're a data or ML person who has always felt slightly fraudulent about the "software" half of the job, this is the scaffolding I wish I'd had. Steal it, drop the stages your work hasn't earned, and make it yours.

References

1. Andrej Karpathy, the original "vibe coding" post (Feb 2025). https://x.com/karpathy/status/1886192184808149383
2. Simon Willison, "Not all AI-assisted programming is vibe coding." https://simonwillison.net/2025/Mar/19/vibe-coding/
3. Addy Osmani, "The 70% problem: hard truths about AI-assisted coding." https://addyo.substack.com/p/the-70-problem-hard-truths-about
4. Spec-Driven Development with AI, The GitHub Blog. https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/
5. Spec Kit, GitHub's spec-driven development toolkit. https://github.com/github/spec-kit

6. Superpowers, Jesse Vincent (obra). https://github.com/obra/superpowers
7. GStack, the review-and-ship skill collection referenced here, from my own Claude Code setup.
8. OWASP Top 10 for LLM Applications (2025). https://genai.owasp.org/llm-top-10/
9. Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena” (2023). https://arxiv.org/abs/2306.05685